# Baseline Results

PID-Comm Reproduction — Group 4

Rishikesh Machamalla (230101060) Aditya Mogili (230101065) Roopesh Jarupula (230101048)

CS5290 Memory-Centric Architectures — Assignment 2

**Scope of this document.** This covers the *baseline* and *main-technique* reproduction only (what the assignment calls "implement the baseline... then the paper's main technique, and compare your trends with the paper's figures"), plus the BFS application used to validate it. Our three new additions (the cost model, the bitwise-reduction finding, and the AVX2 study) are reported separately in the Contribution Results document. Full methodology, every caveat, and the explanation for each disagreement are in the main report; this document presents only the resulting graphs and tables, as requested.

**What "baseline" means here.** We implement the paper's conventional host-mediated path (**L0**) and its three optimizations applied incrementally: **L1** = +PE-assisted reordering, **L2** = +in-register modulation, **L3** = +cross-domain modulation (the full design), for four primitives: AlltoAll, ReduceScatter, AllGather, AllReduce. All numbers are *host-side time of a software model* (Intel i7-1360P, pinned core, minimum of 15 repetitions, measurement noise 2.2% mean) — *not* UPMEM hardware measurements. We could not run the authors' own implementation (it crashed on the available functional simulator), so these come from our independently-verified software model instead.

## 1 Correctness (underlying every number below)

| Check | Result | What it shows |
|---|---|---|
| C++ correctness suite | 17,976 / 17,976 pass | Every primitive, level, shape, dtype, operator |
| Python test suite | 334 pass, 2 skipped | Independent NumPy reference matches the C++ engine |
| Fault-injection test | 37/200 fail when broken | The cross-check genuinely detects bugs (not vacuous) |

## 2 Primary Comparison: Our Speedups vs. the Paper's

64 PEs, 64 KiB/PE, 32-bit sums, scalar kernels (identical kernels at every level, so the algorithm is the only variable). This is the headline comparison — analogue of the paper's Fig. 14 and Fig. 16.

| Quantity (full design L3 vs. baseline L0) | Paper (1024 PEs, real HW) | Ours (64 PEs, software model) |
|---|---|---|
| AlltoAll speedup | 5.19× | 9.53× |
| ReduceScatter speedup | 4.46× | 1.38× |
| AllReduce speedup | 4.23× | 3.32× |
| AllGather speedup | — | 2.39× |
| +PR step alone (geomean) | 1.48× | 0.67×[†] |
| +IM step, additional (geomean) | 2.03× | 2.46× |
| +CM step, additional, AA/AG (geomean) | 1.42× | 3.15× |

[†] Below 1×: PE-assisted reordering alone is *slower* than the baseline in our model (host-side time excludes PE-side work; see report Sec. IV).

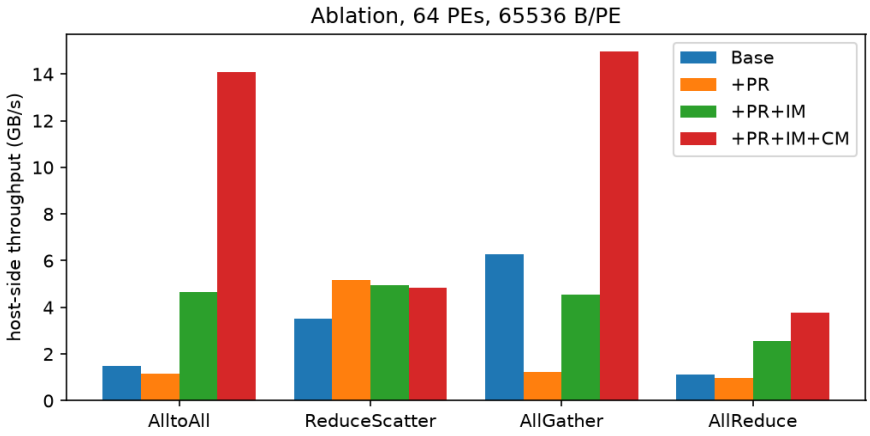



*Figure 1. Ablation: host-side throughput at each implementation level (64 PEs, scalar kernels). From `cpp_backend/results/fig_ablation.png` after running `scripts/run_all.sh`.*

## 3 Sensitivity: Data Size

*Note: only the smallest- and largest-size geomean values were captured in our summary comparison; the full 5-point sweep exists as a figure on the measurement machine (see note below).*

| | 512 B/PE | 65,536 B/PE |
|---|---|---|
| Geomean speedup (L3 vs. L0, 4 primitives) | 2.49× | 3.19× |

Speedup grows with data size, matching the paper's Fig. 18 trend (paper: 2.89× geomean at 8 MB, real hardware).

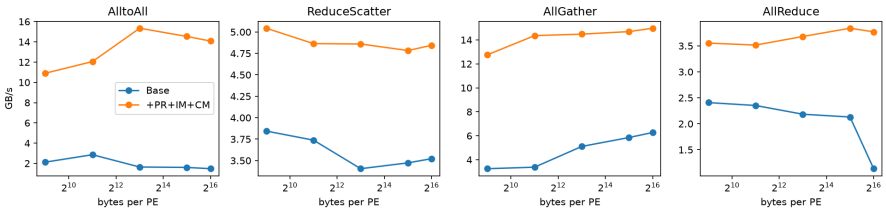



*Figure 2. Throughput vs. data size, all 4 primitives, L0 vs. L3. From `fig_datasize.png`.*

## 4 Sensitivity: Number of PEs

*Note: only the 8-PE and 64-PE geomean values were captured in our summary comparison; the full 4-point sweep exists as a figure (see note below).*

| PEs | 8 | 16 | 32 | 64 |
|---|---|---|---|---|
| Geomean speedup (L3 vs. L0, 4 primitives) | 1.79× | — | — | 3.23× |

Speedup grows with PE count (8→64), matching the paper's Fig. 19 trend (paper: scales to 1024 PEs on real hardware; we are limited to 64).

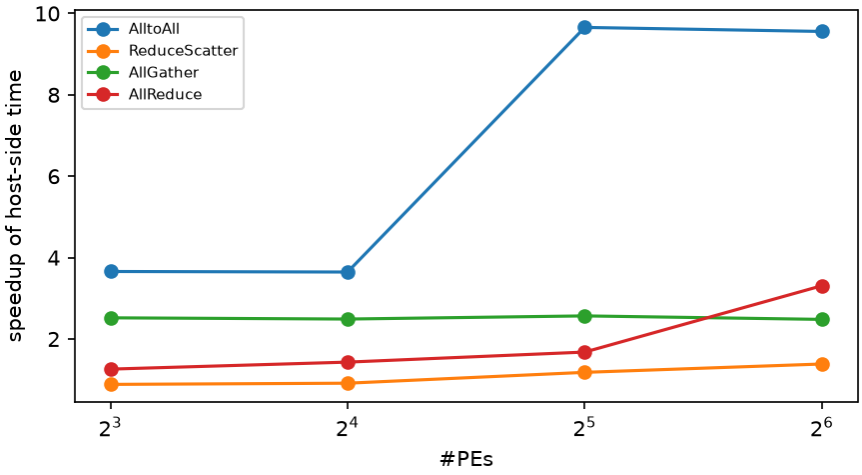



*Figure 3. Speedup vs. number of PEs, all 4 primitives. From* `fig_pe_sweep.png`.

## 5 Sensitivity: Element Width

| Element width | 8-bit | 32-bit | 64-bit |
|---|---|---|---|
| ReduceScatter/AllReduce speedup (L3 vs. L2, geomean) | 2.88× | 2.13× | 1.86× |

Narrower (8-bit) data benefits most, because cross-domain modulation then applies to arithmetic reductions too — matching the paper's claim (Sec. VIII-F).

## 6 Sensitivity: Hypercube Shape

| Primitive | max/min throughput across shapes | |
|---|---|---|
| AlltoAll | 1.08 | (nearly flat, as paper reports) |
| AllReduce | 1.20 | (nearly flat, as paper reports) |

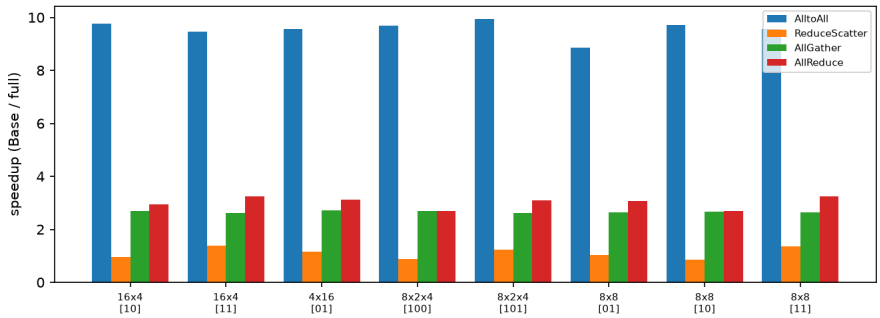



*Figure 4. Throughput across different hypercube shapes/communication masks. From* `fig_shapes.png`.

## 7 Application Baseline: BFS

BFS partitions graph edges over PEs; each round every PE expands its local frontier, then `AllReduce(bitwise OR)` merges the global next frontier — as in the paper. $V$=16,384, $E$=65,532, 8 rounds, 16,380

vertices reached. Correctness verified three independent ways (Python reference, C++ on identical graph files, plain queue BFS); all agree. Only the AllReduce communication is timed — we make no end-to-end speedup claim.

| PEs | L0 (ms) | L1 (ms) | L2 (ms) | L3 (ms) | L0/L3 speedup |
|---|---|---|---|---|---|
| 8 | 0.038 | 0.068 | 0.051 | 0.019 | 2.03× |
| 16 | 0.075 | 0.132 | 0.098 | 0.036 | 2.06× |
| 32 | 0.179 | 0.294 | 0.195 | 0.074 | 2.41× |
| 64 | 0.386 | 0.613 | 0.401 | 0.157 | 2.46× |

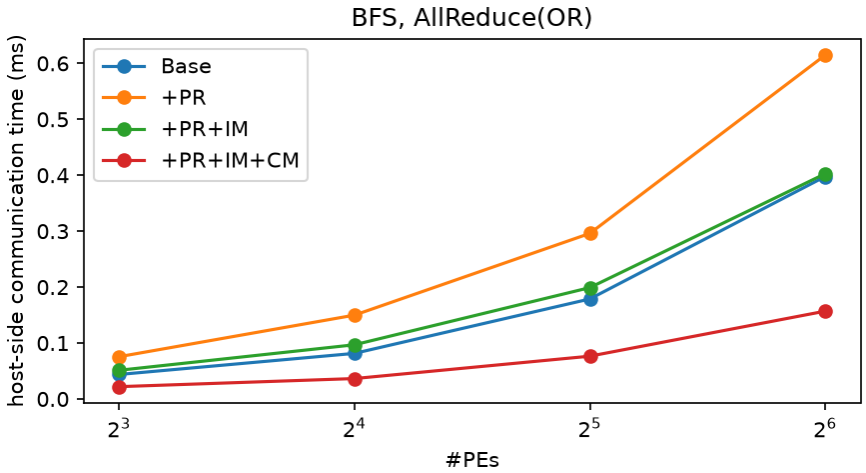



*Figure 5. BFS host-side communication time by level and PE count. From* `results/bfs/fig_bfs.png` *(produced by* `scripts/run_bfs.py`*).*

---

*All measurements are host-side timings of our software model on one laptop, not UPMEM hardware measurements. Where we differ from the paper (ReduceScatter/AllReduce magnitudes, PE-assisted-reordering-alone), the main report states this explicitly and gives (untested) hypotheses — see Sec. IV of* `report.tex`*.*